

E-graphs modulo theories

presented by Sofia Brookie

E-graphs

A useful method for equational reasoning

E-graphs

A useful method for equational reasoning

Applications:

- compilers & optimization
- program synthesis
- hardware design
- (...*many more*)

E-graphs: the core idea

E-graphs are a data structure for compactly representing a bunch of terms and equations between those terms.

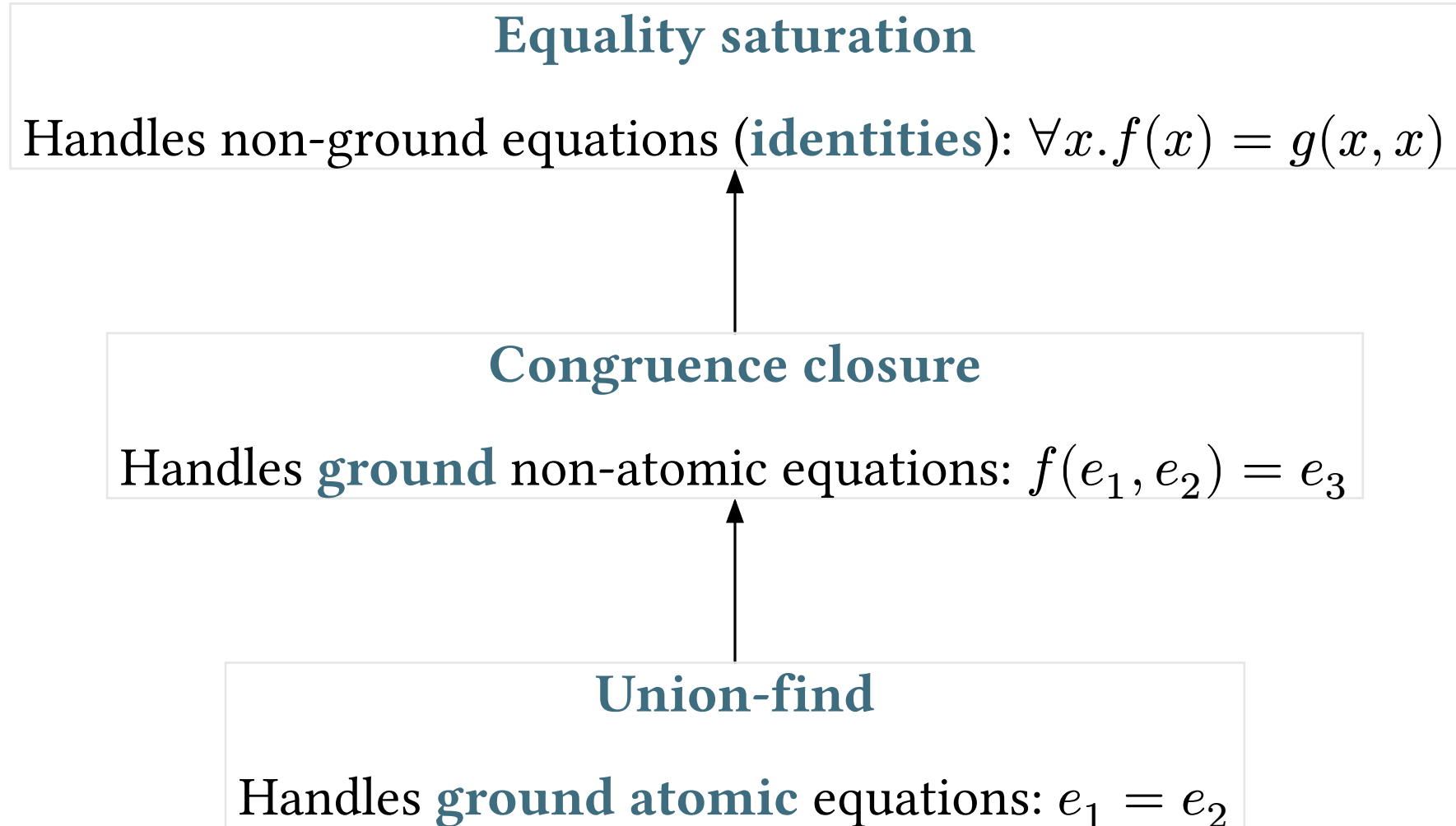
E-graphs: the core idea

E-graphs are a data structure for compactly representing a bunch of terms and equations between those terms.

Egraphs allow operations for things like:

- Looking up terms equal to a given one
- Inserting new terms
- Finding a minimal term equal to another one (according to some cost function)

E-graphs overview

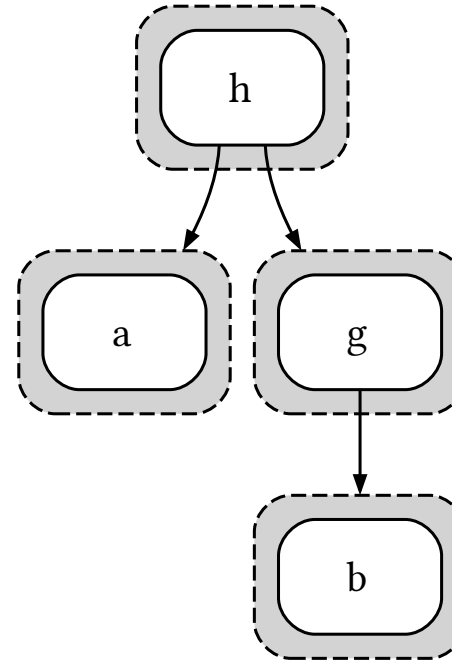


E-graphs basics, part 1: flattening

Let's represent
the term $h(a, g(b))$

White boxes: **E-nodes**

Grey boxes: **E-classes**



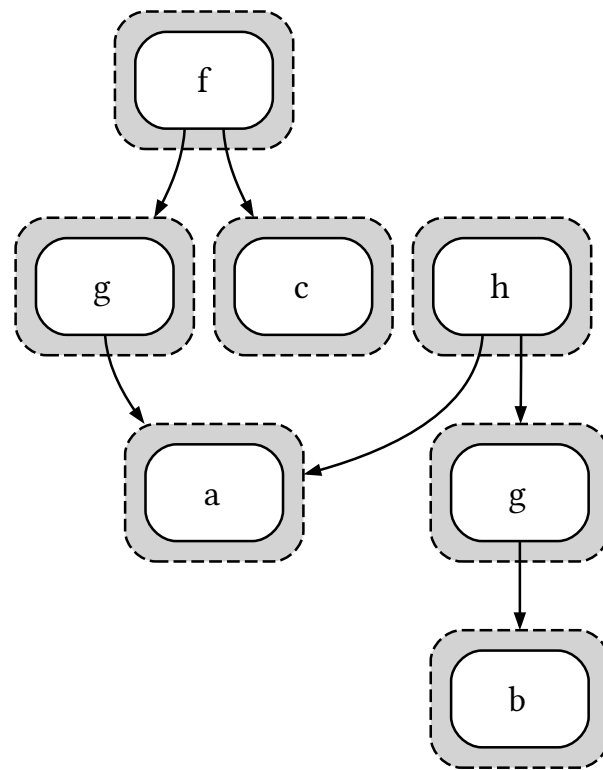
E-graph basics, part 2: sharing

Let's add
the term $f(g(a), c)$

(represented:

$h(a, g(b)),$

$f(g(a), c))$



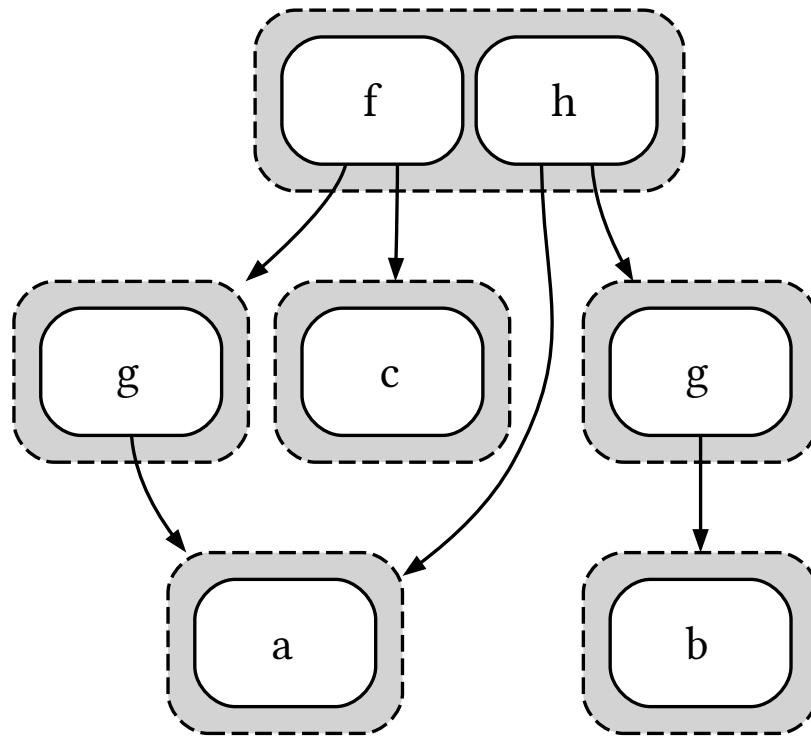
E-graph basic, part 3: equations

Let's add
the **equation**

$$h(a, g(b)) = f(g(a), c)$$

(represented:

$$h(a, g(b)) = f(g(a), c))$$



E-graph basics, part 4: more sharing

Let's add

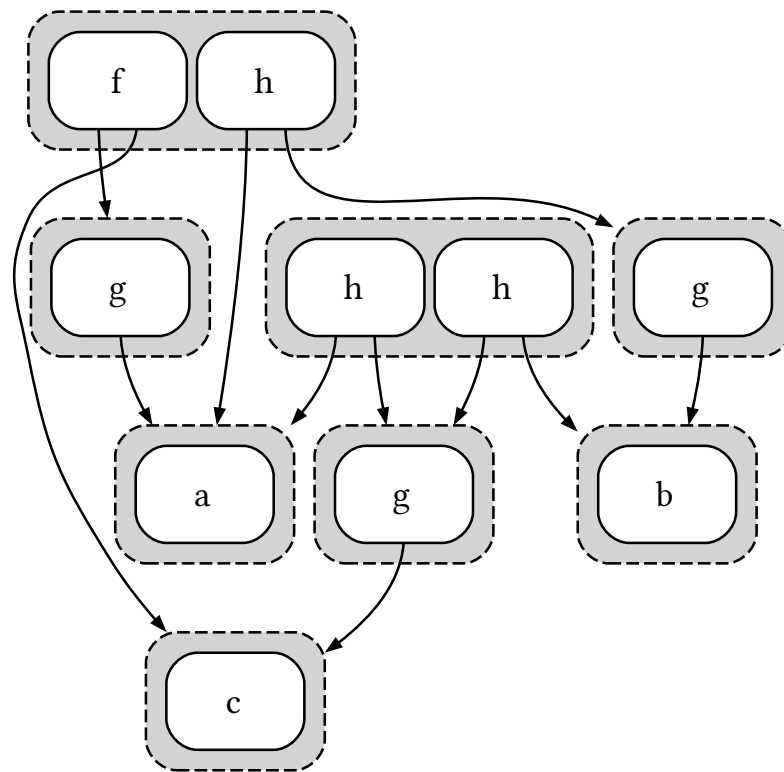
another equation

$$h(g(c), b) = h(a, g(c))$$

(represented:

$$h(a, g(b)) = f(g(a), c)$$

$$h(g(c), b) = h(a, g(c)))$$



E-graph basic, part 5: contraction

Let's add
another equation

$$g(b) = g(c)$$

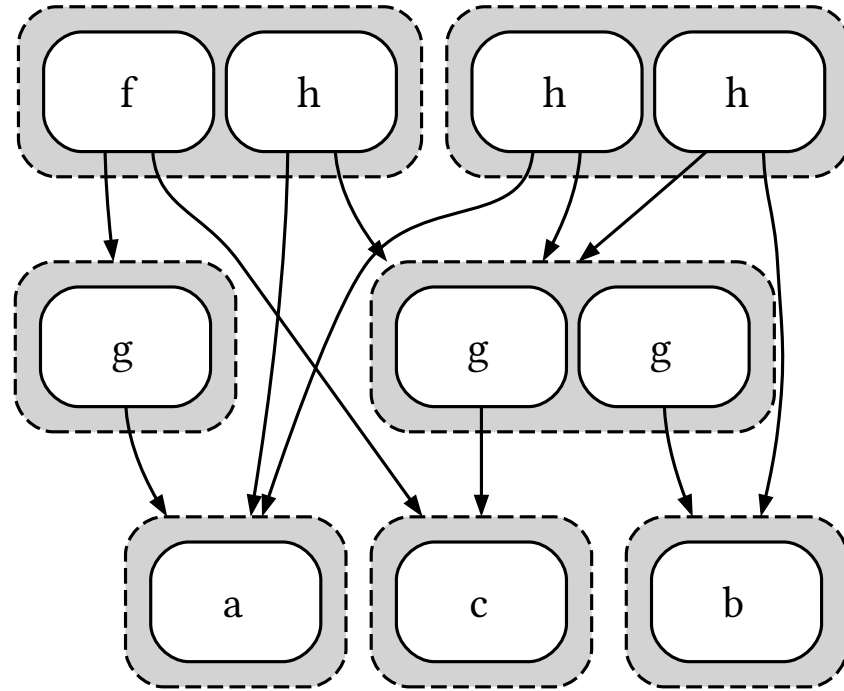
Existing E-classes
are merged

(represented:

$$h(a, g(b)) = f(g(a), c)$$

$$h(g(c), b) = h(a, g(c))$$

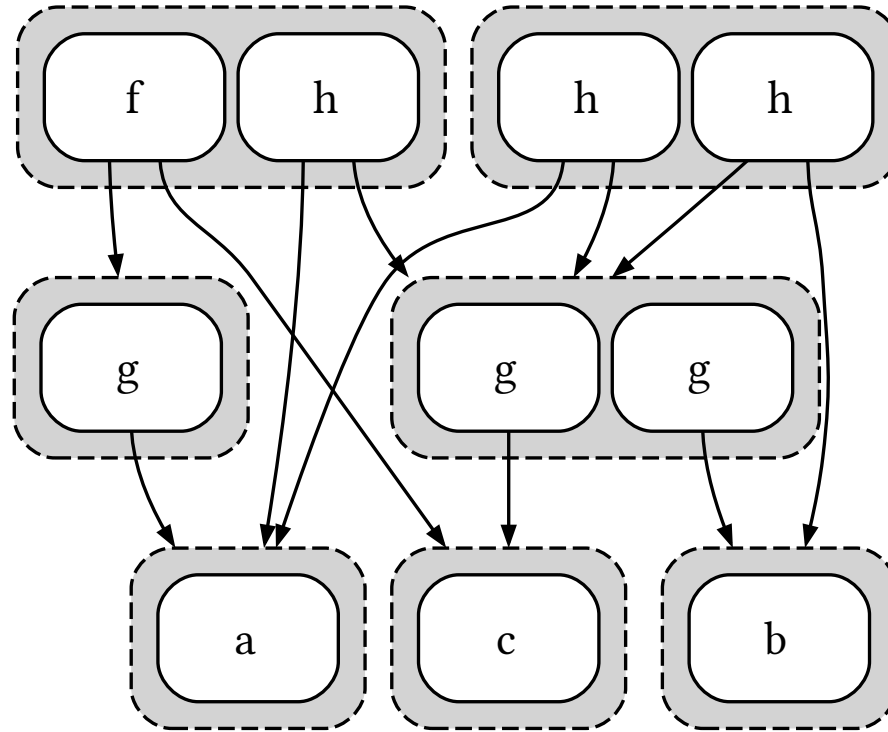
$$g(b) = g(c))$$



E-graph basic, part 6: congruence

Now, $h(a, g(b))$
and $h(a, g(c))$
are both in
the E-graph.

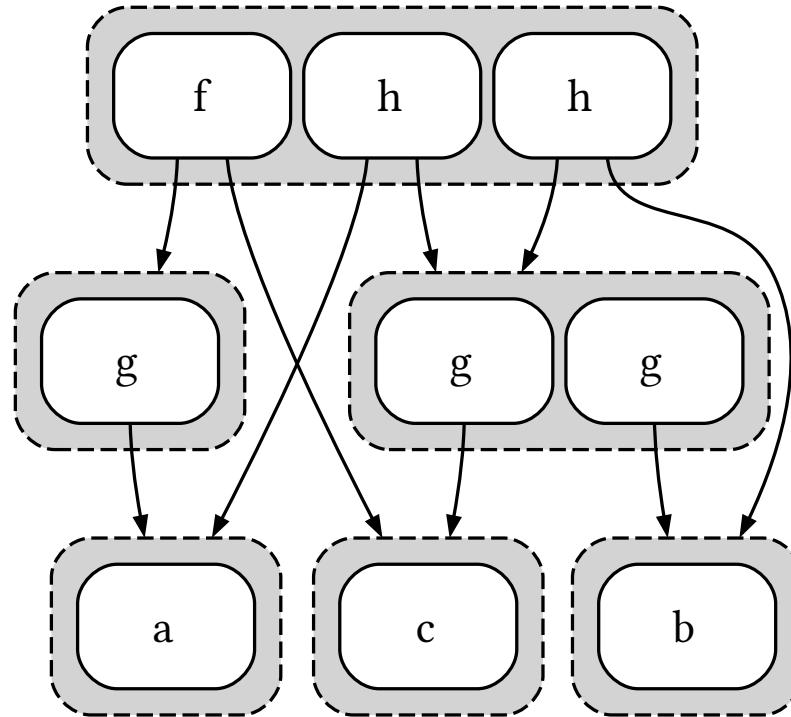
They ought to
be equal, since
 $g(b) = g(c)$



E-graph basic, part 6: propagation

We need to
detect when two
E-nodes have the
same symbol and
same children.

Then, we merge
the E-classes:



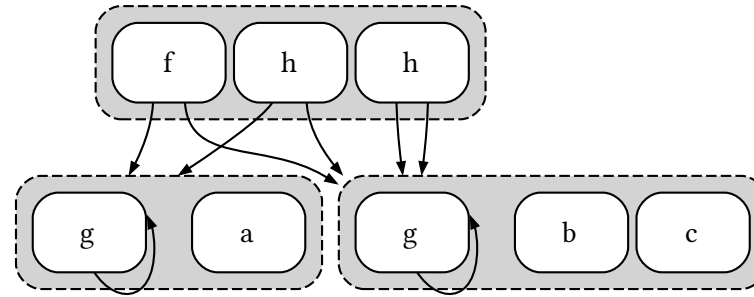
E-graphs: adding equality saturation

Congruence closure decides the theory of ground equations. What about identities?

E-graph basics, part 7: equality saturation

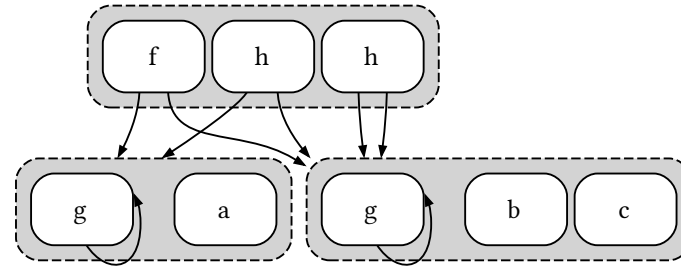
Consider the identity

$$\forall x. g(x) = x$$



E-graph basic, part 8

Note that $g(g(\dots(a))) = a$
is represented implicitly
by this final graph
via **cycles**



Equality saturation

If we have $\forall x..., t = s$, try looking for **instances** of this in the E-graph.

If we find an instance, we might have to add the other side into the E-graph, if it is not yet present. Then, we merge the corresponding E-classes.

Equality saturation

Loop:

- E-match
- Insert (if needed)
- Merge

Equality saturation

This doesn't always terminate

Congruence closure only ever shrinks the E-graph, by bringing together E-classes

Insertion will grow the E-graph

Equality saturation

Still, it terminates in many cases, or we can use heuristics to decide when to stop

E-graphs vs AC

A: associativity

$$f(x, f(y, z)) = f(f(x, y), z)$$

C: commutativity

$$f(x, y) = f(y, x)$$

AC: both

E-graphs vs AC

The bane of equality saturation is operators that are AC or similar (AC + unit, AC + inverse, etc)

We will use ‘+’ as our example.

E-graphs vs AC

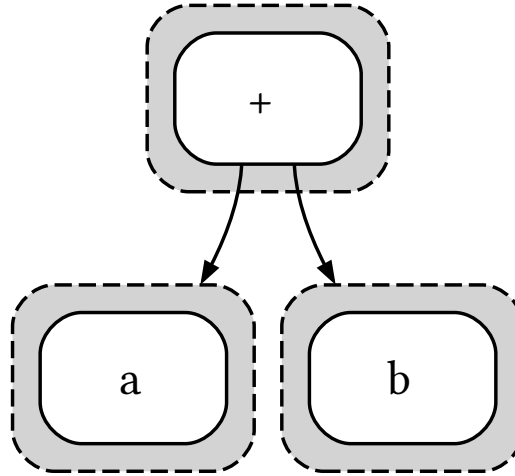


Figure 10: Size 2

E-graphs vs AC

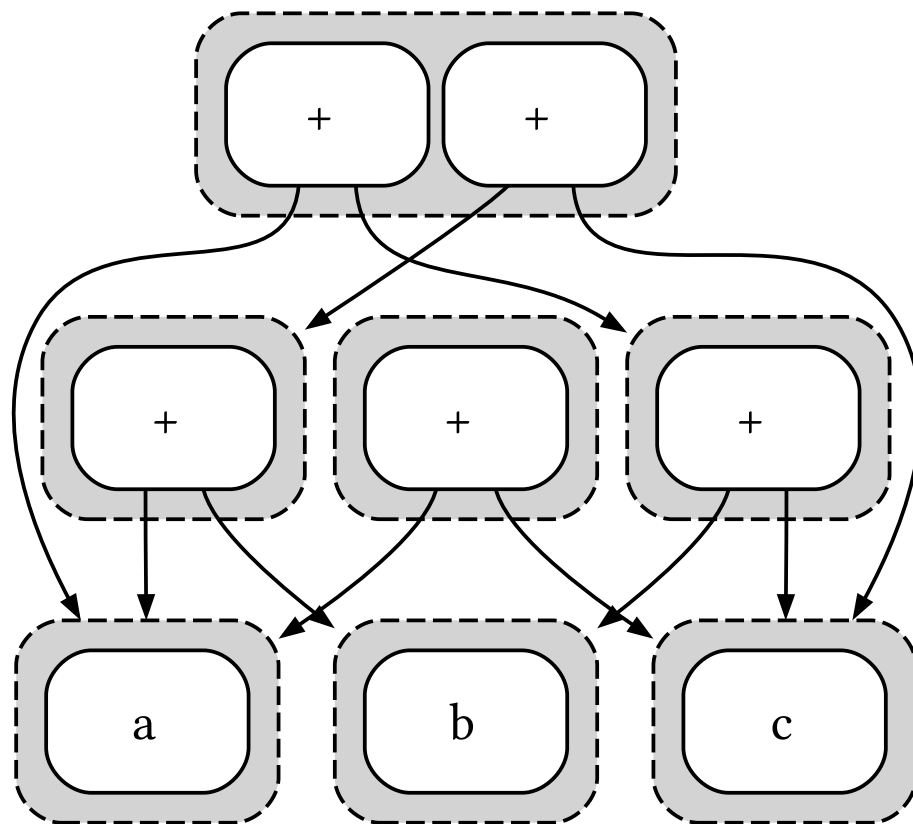


Figure 11: Size 3

E-graphs vs AC

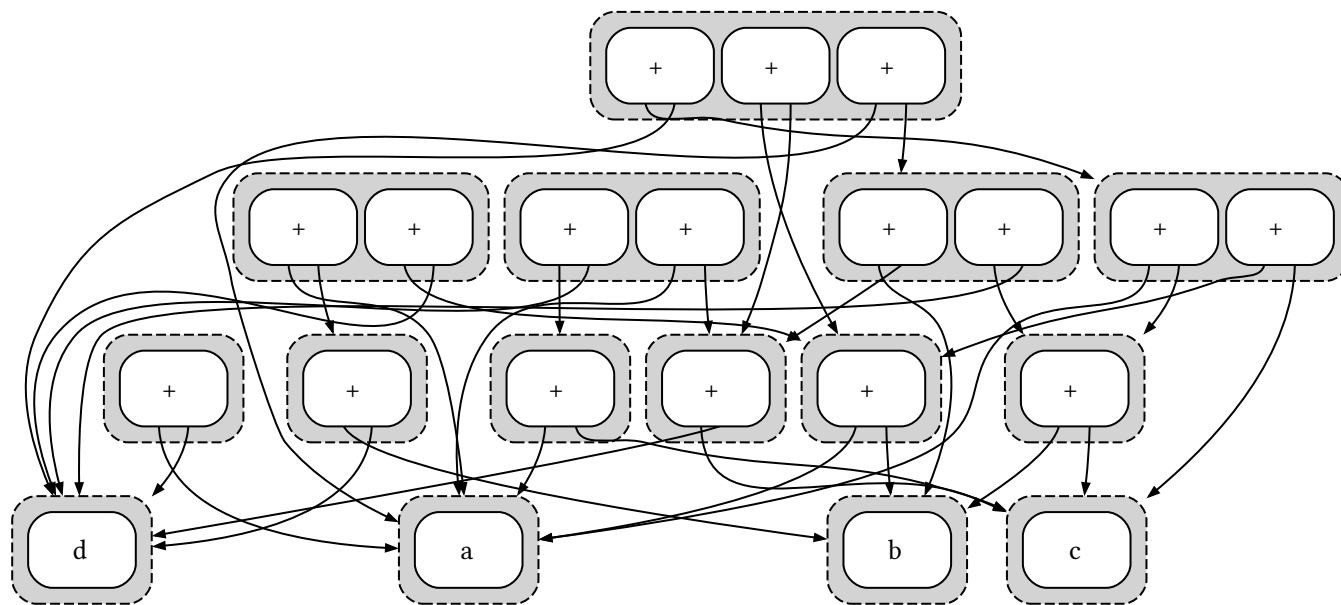


Figure 12: Size 4

E-graphs vs AC

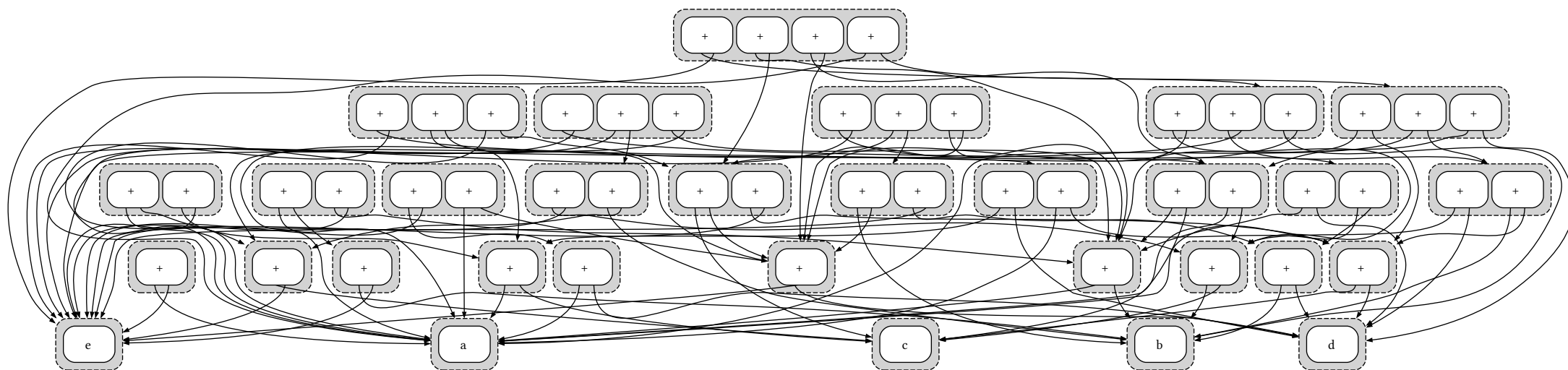


Figure 13: Size 5

E-graphs vs AC

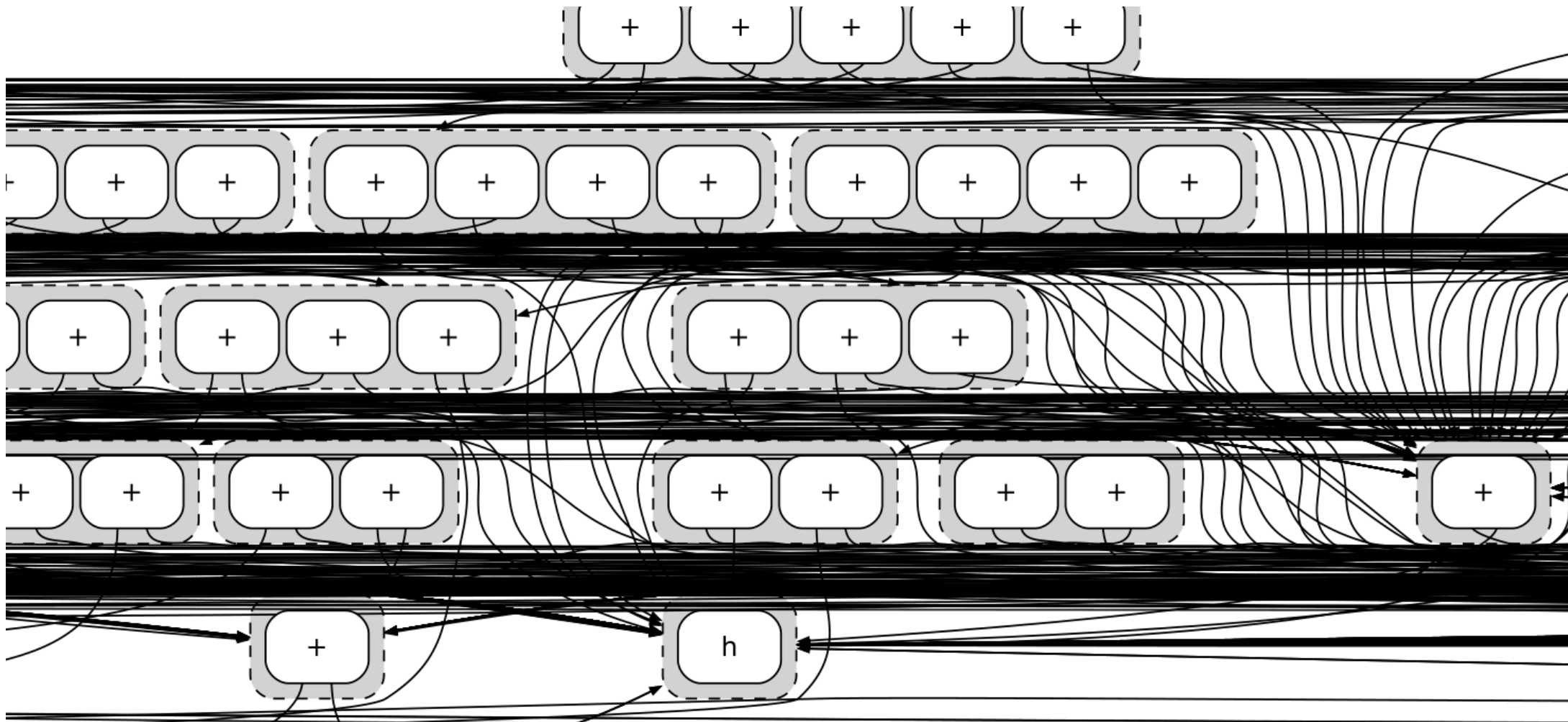


Figure 14: Size 8 (a small window)

E-graphs vs AC

Simply saturating with A and C gives $O(2^n)$ E-classes if you have a starting term with n +s ($a + b + c + \dots$), and each E-class has $O(2^n)$ E-nodes in it! So, **doubly exponential**.

AC problems, an example

Consider an affine function f with identity

$$\forall xy. f(x + y) + 1 = f(x) + f(y)$$

and the starting term $f(a) + f(b) + f(c)$. We can to rewrite this to $f(a + b + c) + 2$

E-graphs vs AC

Using the online Egglog demo:

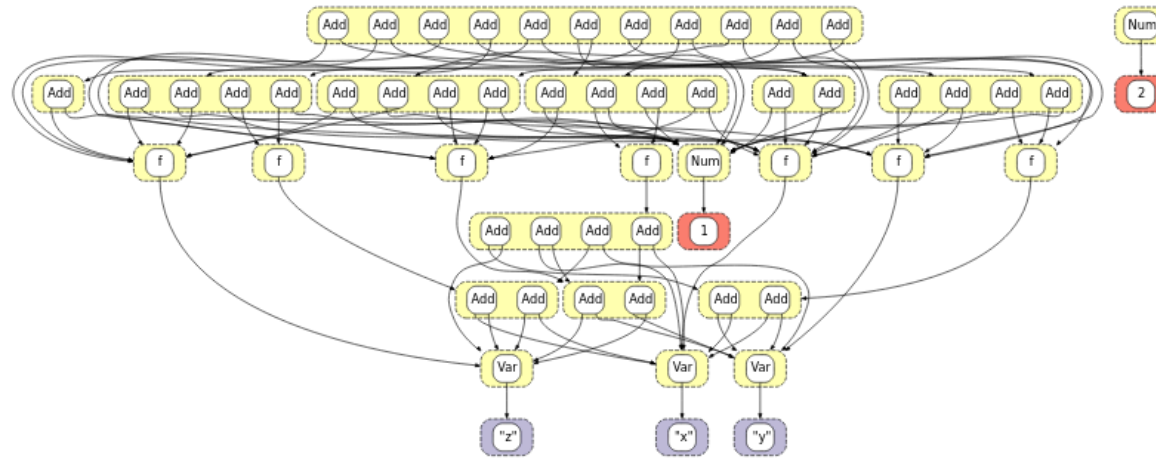


Figure 15: E-graph at timeout

The solution? EMTs

SAT(satisfiability) solvers are a programs that can solve certain logical problems.

They have been extended to SMT (Satisfiability Modulo Theories) solvers, which can solve lots more problems by integrating a bunch of theory-specific solvers into the general logical framework.

This approach has been very successful, and SMT solvers have found endless uses.

Why can't we handle AC in a specific way, rather than relying on equality saturation?

The solution? EMTs

SAT(satisfiability) solvers are a programs that can solve certain logical problems.

They have been extended to SMT (Satisfiability Modulo Theories) solvers, which can solve lots more problems by integrating a bunch of theory-specific solvers into the general logical framework.

This approach has been very successful, and SMT solvers have found endless uses.

Why can't we handle AC in a specific way, rather than relying on equality saturation?

This is a natural idea, that has been thrown around for a while.

EMT example

We think of +-nodes
'up to AC'.

So, they are variadic
with **unordered**
children

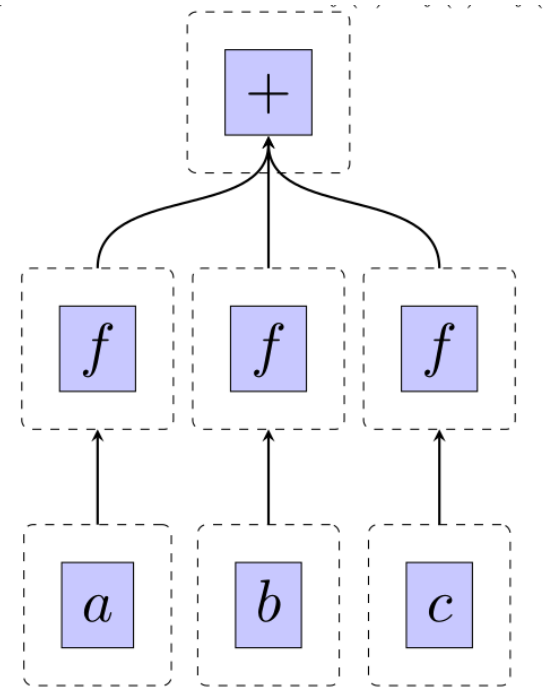


Figure 16: EMTs to the rescue...

EMT example

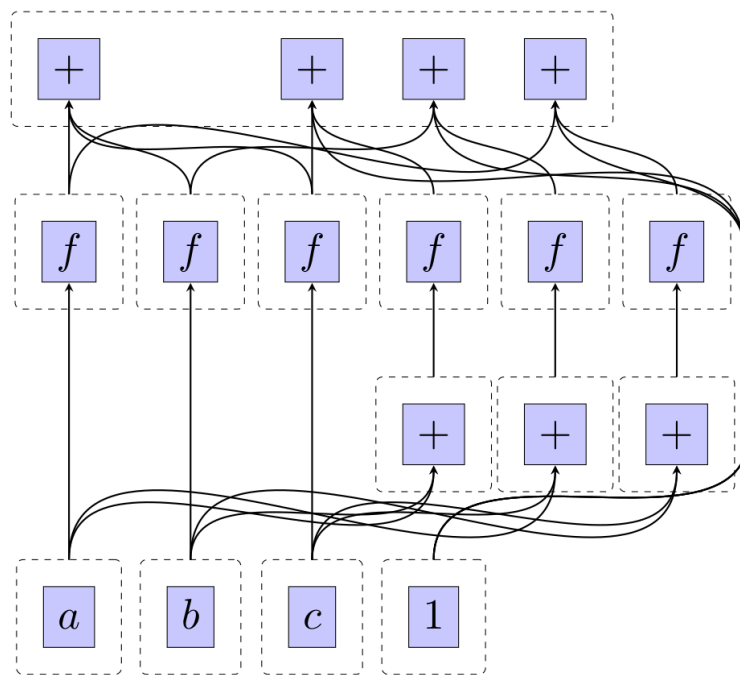


Figure 17: EMTs to the rescue...

The solution??

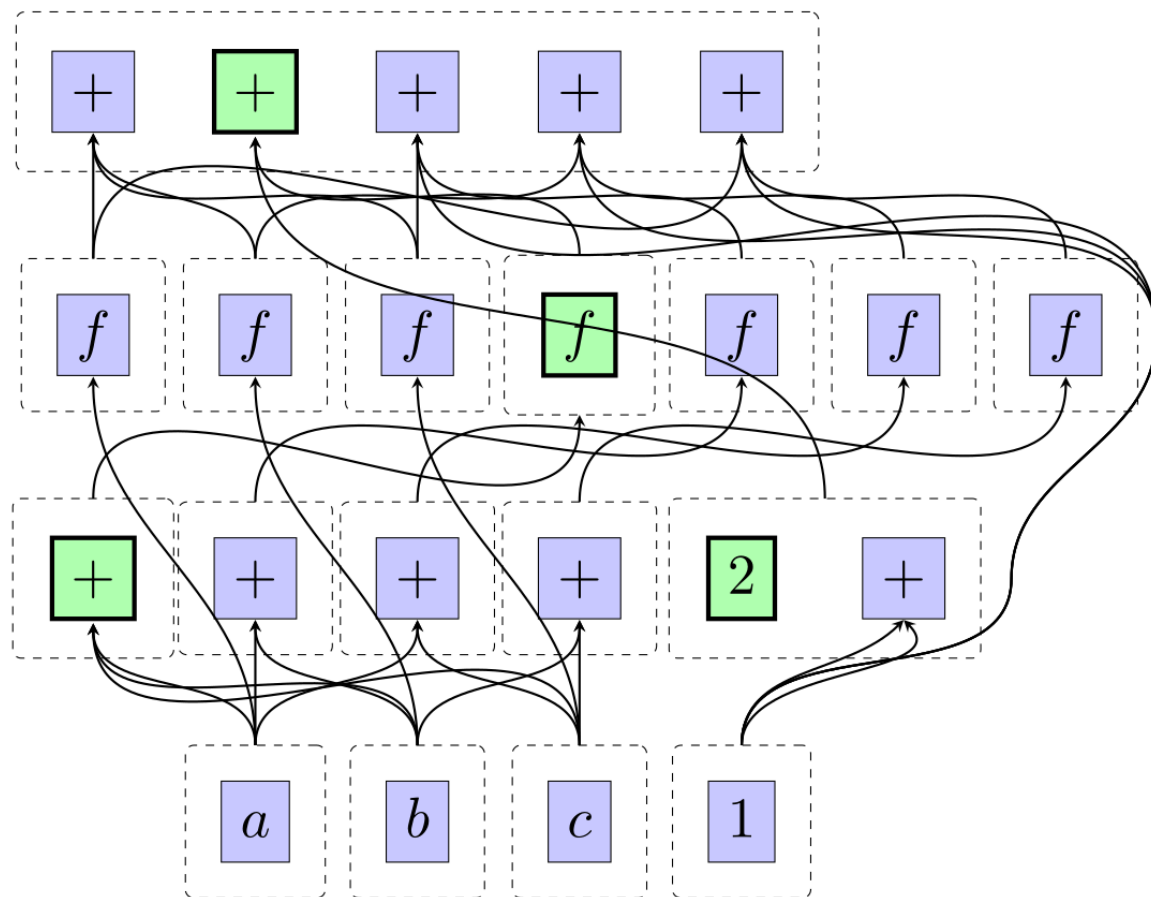


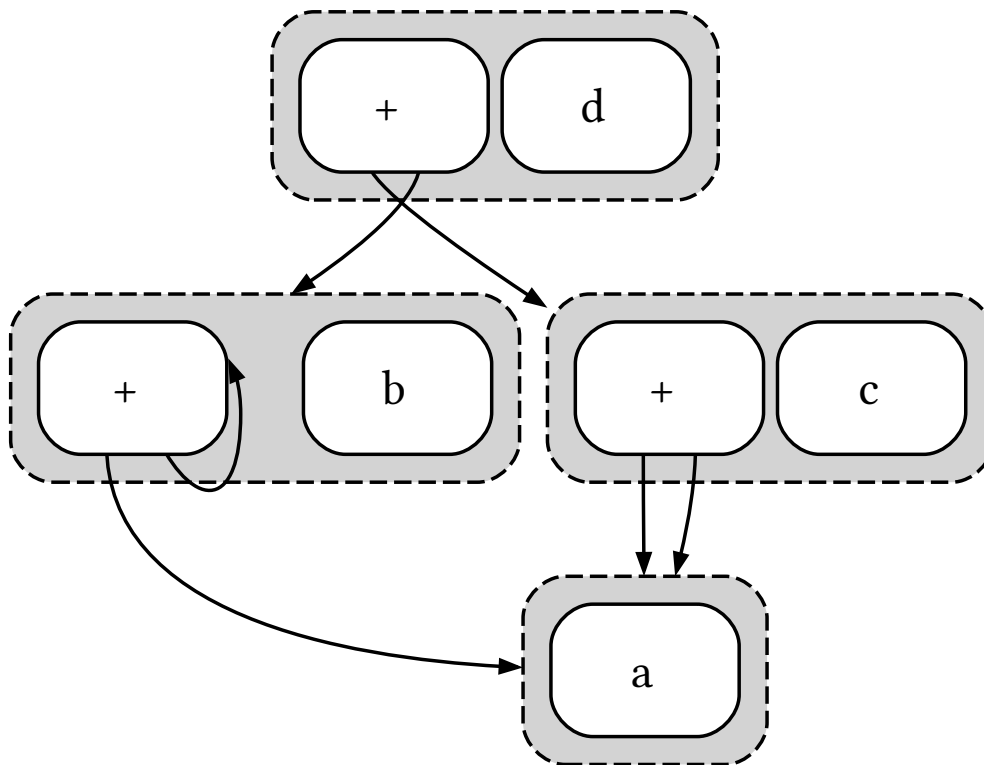
Figure 18: EMTs to the rescue...

Gaps

$$a + b = b$$

$$a + a = c$$

$$c + b = d$$



Gaps

$$(a + b = b)$$

$$(a + a = c)$$

$$(c + b = d)$$

b

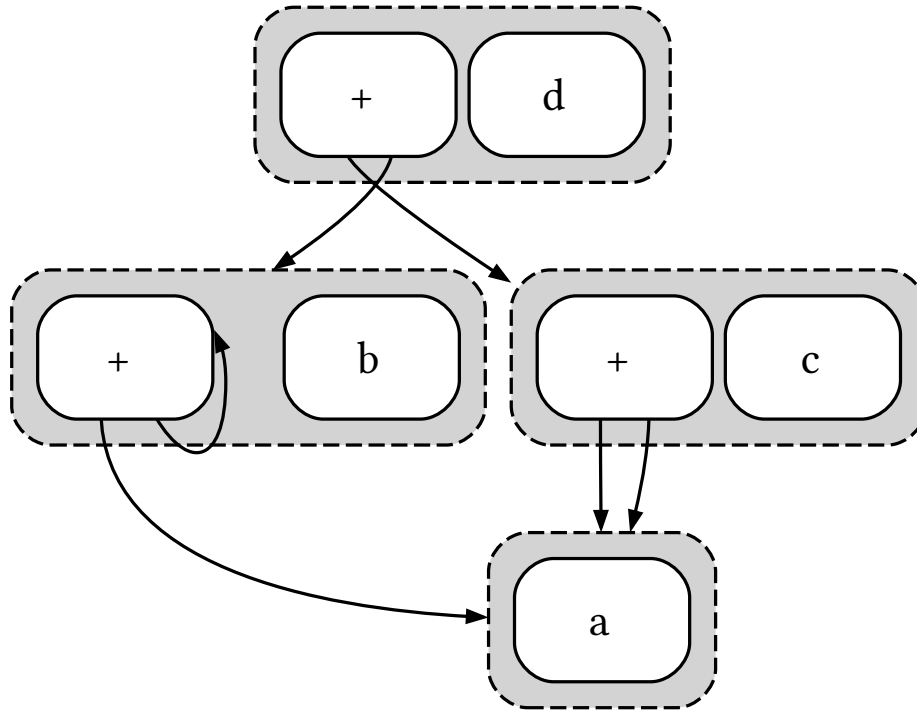
$$= a + b$$

$$= a + (a + b)$$

$$= (a + a) + b$$

$$= c + b$$

$$= d$$



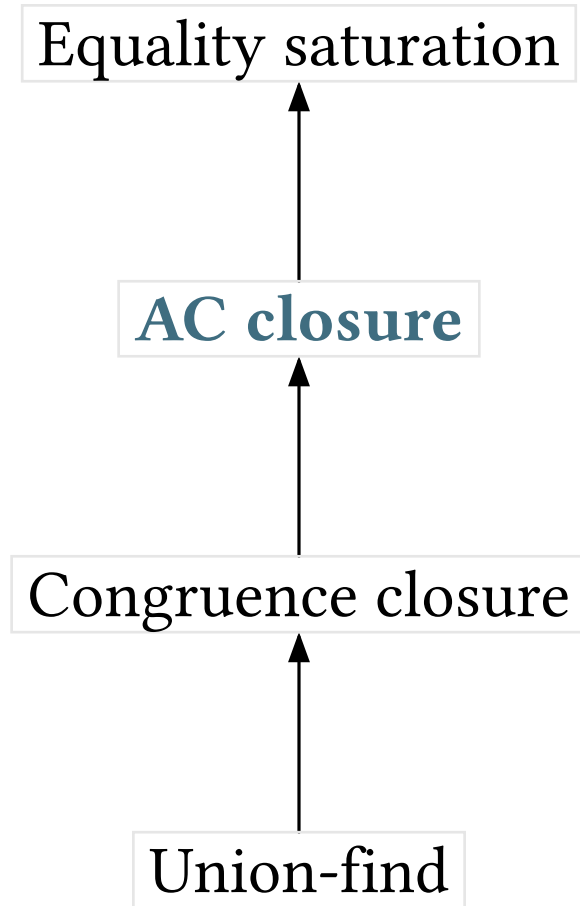
The problem

Just making nodes work “up to AC” has lost the equational reasoning required to properly handle AC.

We have an E-graph closed under congruence, but it’s missing the desired equation.

We could add back saturation on the AC identities, but that unsolves the problem we were trying to solve in the first place.

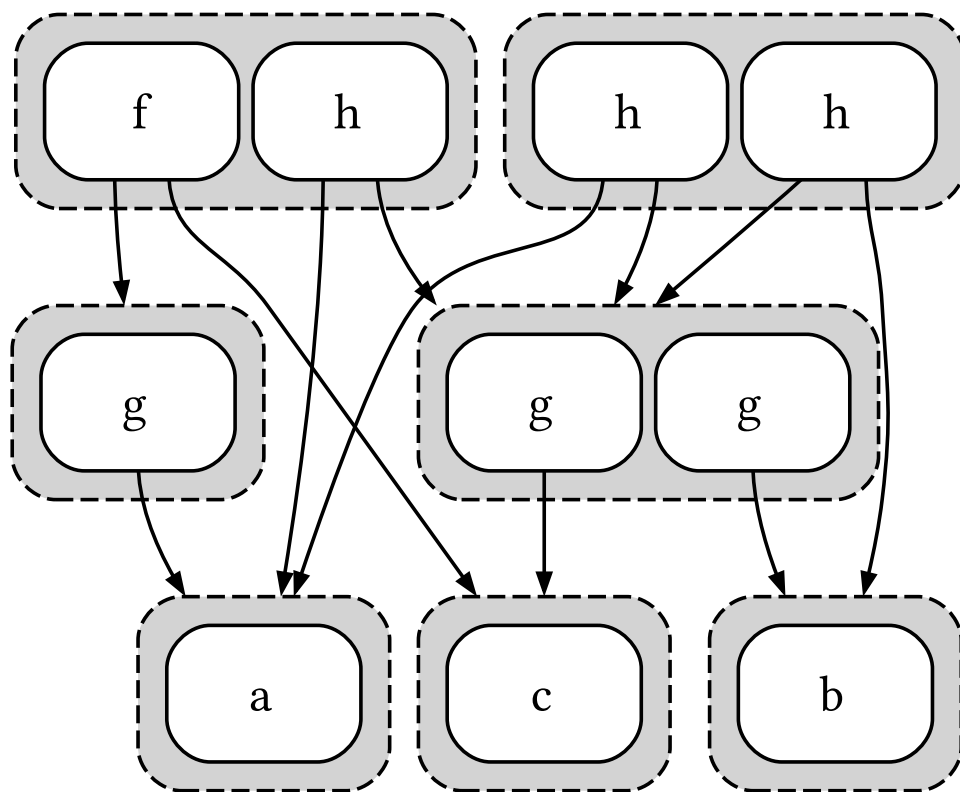
Making EMTs work: our contribution



Some insights from rewriting

E-graphs are just certain kinds of rewrite systems

E-graphs as a rewrite system



$$a \rightarrow e_a$$

$$b \rightarrow e_b$$

$$c \rightarrow e_c$$

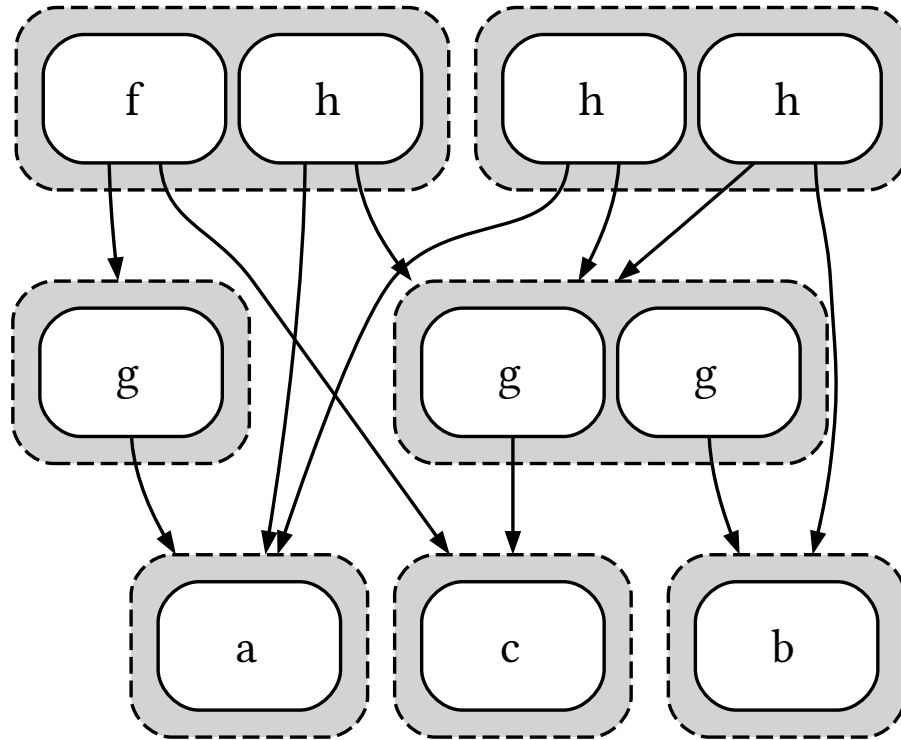
$$g(e_a) \rightarrow e_1$$

$$g(e_b) \rightarrow e_2$$

$$g(e_c) \rightarrow e_2$$

...

E-graphs as a rewrite system



$$f(e_1, e_c) \rightarrow e_3$$

$$h(e_a, e_2) \rightarrow e_3$$

$$h(e_a, e_2) \rightarrow e_4$$

$$h(e_2, e_b) \rightarrow e_4$$

What congruence closure is doing

$$e_3 \leftarrow h(e_a, e_2) \rightarrow e_4$$

When we can rewrite in two different ways, this is called a **critical pair**.

Resolve critical pairs by merging: in this case, e_3 and e_4

AC closure

We need to find AC-critical pairs

$$c + b \leftarrow (a + a) + b \stackrel{=}{\underset{\{AC\}}{}} a + (a + b) \rightarrow a + b$$

So, we learn that $c + b = a + b$

AC closure

Loop:

- Find an AC-critical pair $t_1 \leftarrow s \rightarrow t_2$
- Take the equation $t_1 = t_2$ and **orient** it into a new AC-rewrite rule
 $t_1 \rightarrow t_2$ or $t_2 \rightarrow t_1$
- Interreduce the new rule with the old ones

Why this terminates

Consider the AC-term $a + a + b$ that is on the left hand side of some AC-rewrite rule.

We can think of it as $2a + b$

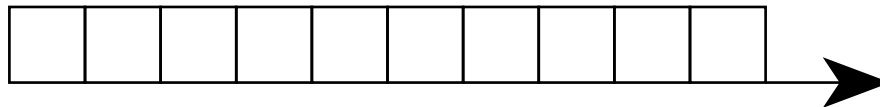
Any new term $k_1 a + k_2 b + \dots$ with $k_1 \geq 2, k_2 \geq 1$ will reduce by this one.

Why this terminates

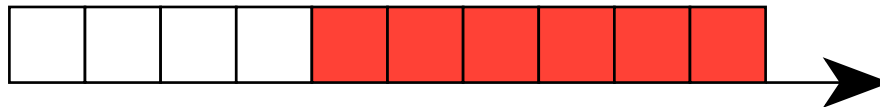
Every rewrite rule means we won't make new rewrite rules that **dominate** it, by having an LHS greater than its LHS in every term: those would get reduced by it.

You can only add so many rewrite rules that aren't dominated by previous ones

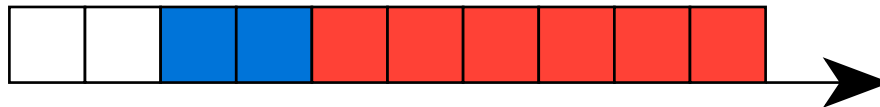
Visual proof



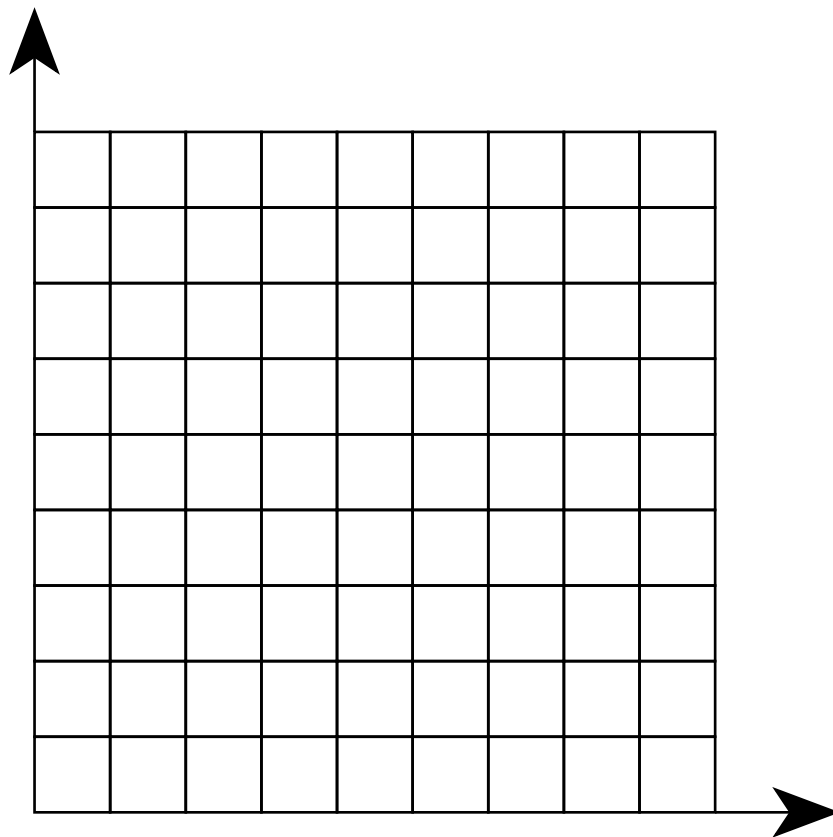
Visual proof



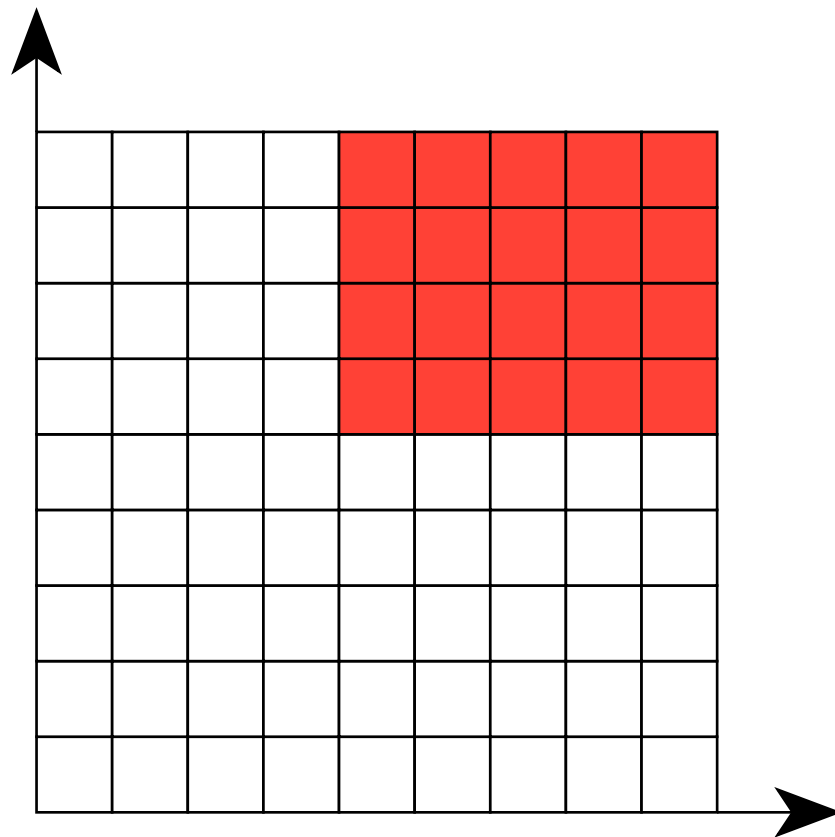
Visual proof



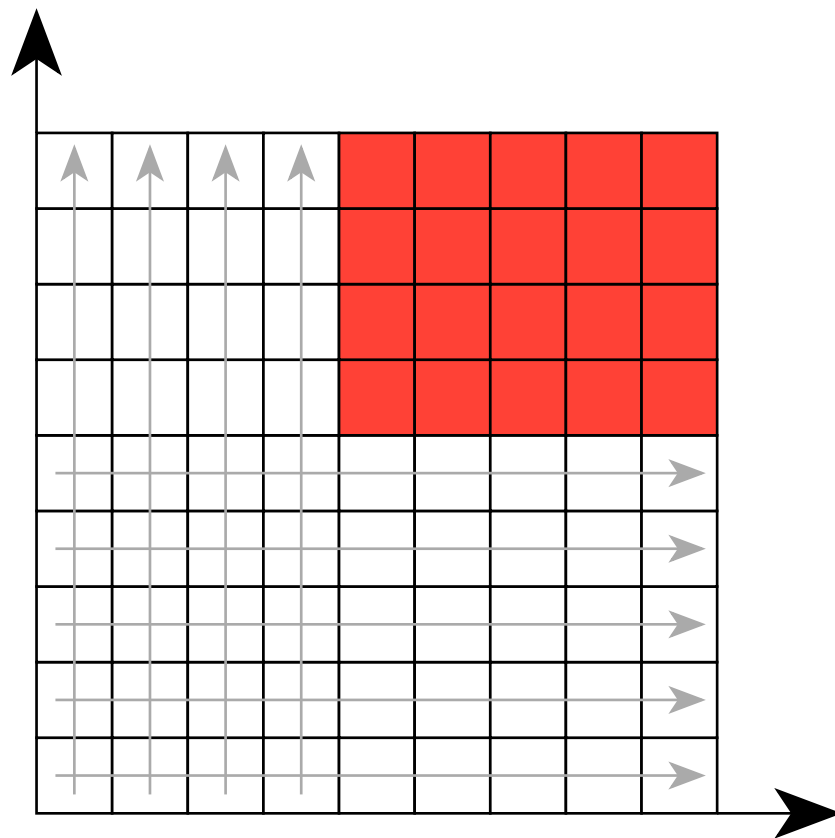
Visual proof



Visual proof



Visual proof



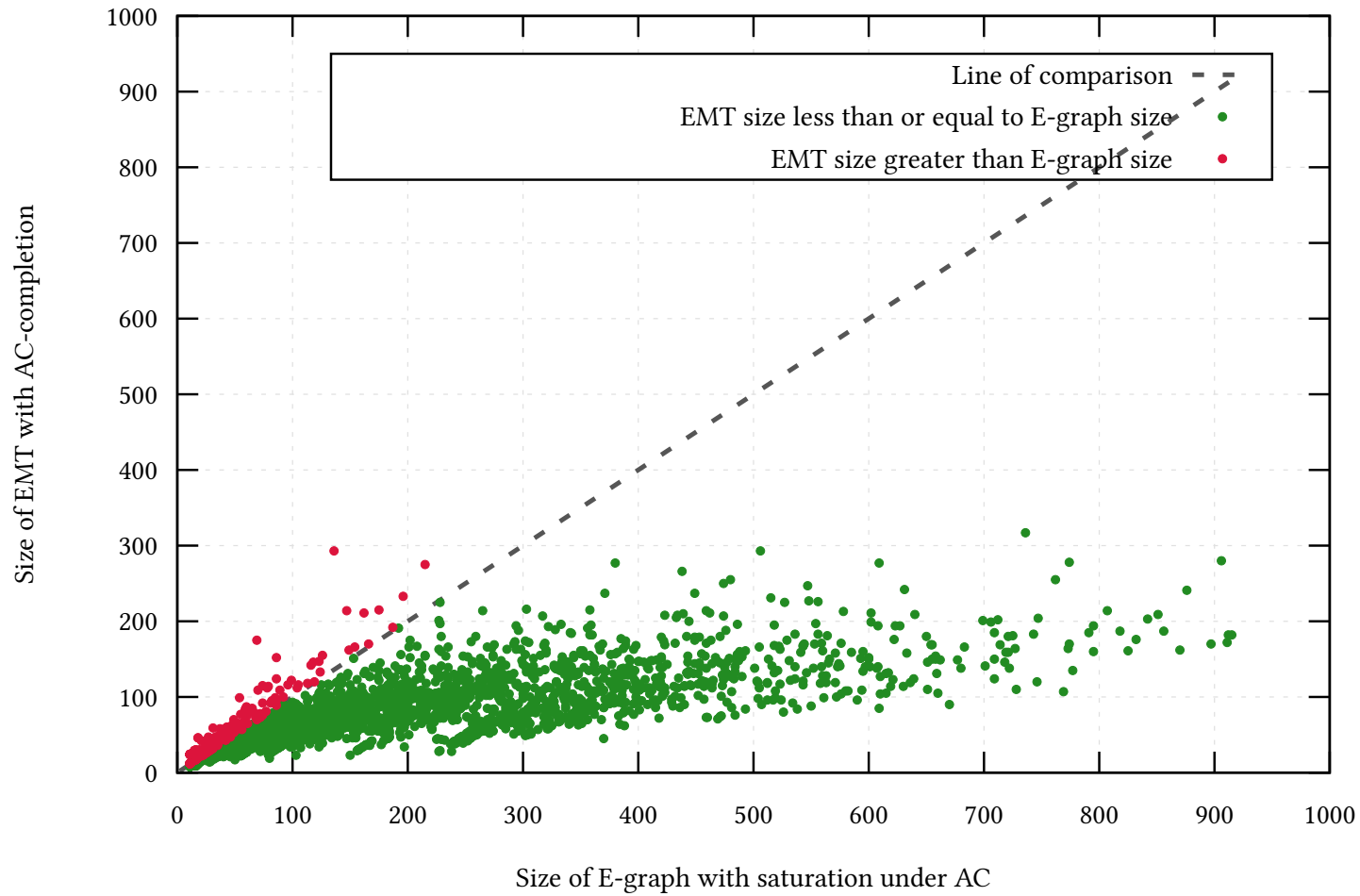
Results

Generate a bunch of random identities (e.g. $f(x) = h(x, f(x))$) and random initial equations.

Run to saturation, handling AC either via EMTs or via saturating on the AC identities

Measure the final size (size of everything, essentially)

Result



AC-closure and the wider world

AC-closure is just a form of **Buchberger's algorithm**, a well-known algorithm in algebra.

Lots of work has been done to optimize it.

Still, the problem is fundamentally ESPACE-complete.

EMTs for any T

Filling the “gap” is exactly equivalent to a well-known problem, the **word problem**

For just associativity, the word problem is undecidable! So, we couldn't just run “A-closure”: we would have to allow the option to run it up to some number of steps, with no guarantees it will ever be done.

Some word problems are much better than ESPACE:

- Abelian groups/ACUN: polynomial time
- Fields, vector spaces, etc: $O(n^{2.\text{something}})$

For AC + other identities (semirings, rings, modules, ...), often a variant of Buchberger's algorithm suffices.